AEM Complete Course

Adobe Experience Manager

Sessions 1 – 9 · Deep Dive Reference

This document is a complete reference for the AEM developer course, covering Sessions 1 through 9. Every session explains concepts from scratch in beginner language with real-world examples, complete annotated code, and step-by-step instructions.

Session 1	CMS, WCM, AEM Architecture, JCR, WYSIWYG, all interfaces
Session 2	Maven, POM.xml, project setup, deploy commands, errors
Session 3	Authoring, Templates, Layout Container, Policies, Sling resolution
Session 4	Custom Components, HTL, Dialogs, EditConfig, Sling Models
Session 5	Touch UI Dialogs, Granite UI, Data Source, Multifield
Session 6	HTL / Sightly — complete reference, Use API, all block statements
Session 7	Sling Models — every annotation, lifecycle, patterns, testing
Session 8	Static & Editable Templates, Page Component, Page Properties
Session 9	Client Libraries — dependencies, embed, minify, gzip, debug

Session 1

CMS, AEM Architecture & Interfaces

What AEM is, the 3-layer setup, JCR, WYSIWYG and every interface

The Problem Before CMS

Before Content Management Systems, every website change required a developer. Updating an interest rate on 47 pages meant SSH-ing into a server, grep-ping HTML files, editing each manually, and FTP-ing them back. A three-hour job for a text change.

Three big problems this caused:

- Developer bottleneck — content editors couldn't self-serve
- Inconsistency — 200 developers editing HTML = 200 different styles over time
- No workflow — no draft state, no approvals, no audit trail

What is CMS vs WCM

CMS (Content Management System) separates content from code. Editors write in a UI; the CMS injects content into templates and renders HTML. Examples: WordPress, Drupal, Contentful.

WCM (Web Content Management) is a CMS built for enterprise scale — hundreds of sites, dozens of languages, strict approval workflows, multi-brand governance. AEM is Adobe's WCM platform.

AEM Technology Stack

AEM is assembled from well-known Apache open-source projects, layered bottom-up:

- **JVM** — AEM is a Java application, started with `java -jar aem-author-p4502.jar`
- **Apache Felix** — the OSGi container. Manages bundles (JARs) at runtime; hot-deploy without restart
- **Apache Jackrabbit Oak** — the JCR (Java Content Repository). Tree-structured storage for all data
- **Apache Sling** — the web framework. Maps URLs to JCR nodes; finds rendering scripts
- **Adobe Granite** — AEM's layer. Adds the authoring UI, workflows, replication, DAM

The 3-Layer Architecture

Every AEM installation always has three separate layers:

- **Author** (port 4502) — internal only; editors create and approve content
- **Publish** (port 4503) — activated content visible to end-users
- **Dispatcher** (port 80/443) — Apache module that caches HTML, blocks dangerous URLs, load-balances

> ***Note:*** *When an editor clicks Publish, AEM serialises the JCR nodes into a package and POSTs it to Publish's replication endpoint. The Dispatcher then receives a flush request to delete the cached HTML for that page.*

The JCR — Java Content Repository

Everything in AEM is a node in a tree. Pages, components, configs, users, assets — all stored as JCR nodes with typed properties.

- `cq:Page` — a web page node
- `cq:Component` — a component definition node
- `nt:unstructured` — a generic data node (dialog values, component content)
- `sling:resourceType` — the most important property: tells Sling which component renders a node

Key JCR paths:

- `/apps/` — your application code (components, templates, clientlibs)
- `/content/` — page content authored by editors
- `/conf/` — editable templates, CF models, site configs
- `/libs/` — AEM's built-in code. **Never edit.** AEM overwrites on Service Pack upgrades

AEM Interfaces

Every AEM interface and its URL:

- **Sites console** — `/sites.html/content` — browse and manage all pages
- **Page editor** — `/editor.html/content/mysite/en/home.html` — WYSIWYG editing
- **CRXDE Lite** — `/crx/de` — browse and edit the JCR directly
- **OSGi console** — `/system/console/bundles` — manage Java bundles
- **Package Manager** — `/crx/packmgr` — install and export JCR content
- **Tools menu** — Templates, Tags, Users, Replication, Cloud Services

Session 2

Maven & Project Setup

Build tool, POM.xml, archetype, project structure, deploy commands

Why Maven

Before Maven, Java development meant manually downloading every library JAR, managing version conflicts, writing giant classpath commands for every compile, and packaging deployments by hand. Maven replaces all of that with one XML file and one command.

> ***Note:*** *The npm analogy: pom.xml is package.json. mvn install is npm install. Maven Central is the npm registry.*

Maven's Four Jobs

- **Dependency management** — declares what libraries you need; Maven downloads exact versions from Maven Central
- **Standardised structure** — `src/main/java/`, `src/test/java/` — every Maven project looks the same
- **Build lifecycle** — validate → compile → test → package → verify → install → deploy (always in order)
- **Plugin execution** — bnd plugin creates OSGi bundles; content-package plugin installs ZIPs to AEM

pom.xml — Key Elements

- **GAV coordinates** — groupId + artifactId + version — the unique identity of every artifact
- **dependency scopes** — `provided` (AEM has it — most AEM APIs), `compile` (bundle it), `test` (JUnit/Mocks only)
- **bnd plugin** — MUST include `Sling-Model-Packages: com.mysite.core.models` or data-sly-use returns null silently
- **Profiles** — `-PautoInstallPackage` uploads the built ZIP to AEM Package Manager API

> ***Watch out:*** *Never use compile scope for AEM/Sling APIs — causes ClassCastException at runtime because the same class loads from two ClassLoaders.*

Project Structure (from Archetype)

- `core/` — Java code (Sling Models, Services, Servlets) → OSGi bundle .jar
- `ui.apps/` — component HTL + dialog XML + clientlibs → content package .zip
- `ui.content/` — sample pages and /conf templates → content package .zip
- `ui.config/` — OSGi .cfg.json files → content package .zip
- `ui.frontend/` — SCSS + TypeScript (webpack) → compiled into ui.apps clientlibs
- `all/` — master installer; embeds all other modules; always built last

Archetype Command

```
mvn -B archetype:generate \
-D archetypeGroupId=com.adobe.aem \
-D archetypeArtifactId=aem-project-archetype \
-D archetypeVersion=48 \
-D appTitle="My Site" \
-D appId="mysite" \
-D groupId="com.mysite" \
-D aemVersion="6.5.0"
```

Deploy Commands

- `mvn clean install -PautoInstallPackage` — full deploy to Author 4502
- `mvn clean install -PautoInstallBundle -pl core` — Java only (fastest for Sling Model changes)
- `mvn clean install -PautoInstallPackage -pl ui.apps` — HTL/dialog changes only
- `mvn clean install -PautoInstallPackage -DskipTests` — skip unit tests
- `mvn clean install -PautoInstallPackage -Daem.host=staging.mysite.com` — remote server

Session 3

Authoring, Templates & Sling Resolution

Page creation, Layout Container, policies, components, Sling URL resolution

Page Authoring Workflow

Sites console → Create Page → pick template → open editor → drag components → configure each (wrench icon, fill dialog, checkmark) → Preview → Publish.

Every dialog save writes JCR properties to the component's content node. The Sling Model reads them. The HTL renders them. The browser displays them.

Static vs Editable Templates

- **Static templates** — stored in `/apps/mysite/templates/`. Developer-only changes. `jcr:primaryType=cq:Template`. One `.content.xml` sets title, allowedPaths, ranking, and the page component path.
- **Editable templates** — stored in `/conf/mysite/settings/wcm/templates/`. Created/edited in AEM UI by template authors. Three layers: Structure (locked), Initial Content (pre-populated), Policies (rules per container).

> ***Note:*** *cq:allowedTemplates on /content/mysite/jcr:content controls which templates appear in the page creation picker.*

Layout Container & 12-Column Grid

The Layout Container (responsive grid) is the droppable area on pages. Editors resize components using the 12-column system in Layout mode. Common combinations: 6+6 (two equal), 8+4 (main+sidebar), 4+4+4 (three equal).

Generated HTML classes: `aem-Grid`, `aem-GridColumn--default--8`, `aem-GridColumn--phone--12` (full-width on mobile).

Policies

Policies are the rules engine for templates. Each Layout Container on a template can have a policy controlling:

- Which components editors can add (Allowed Components)
- Style System CSS class options per component type
- Design dialog values (heading levels, max items, etc.)

Policies are stored under `/conf/mysite/settings/wcm/policies/`. Multiple templates can share the same policy — change once, update everywhere.

Sling URL Anatomy & Resource Resolution

```
http://localhost:4502/content/mysite/en/home.article.wide.html/suffix/data
|_____________________________| |___________| |___| |_____|
JCR path selectors ext suffix
```

Resolution steps: (1) Find JCR node at path. (2) Read `sling:resourceType`. (3) Look for script named `[component].html` in `/apps` first. (4) Walk up `sling:resourceSuperType` chain. (5) Fall back to `/libs`.

> ***Note:*** *sling:resourceType is the most important JCR property. Debugging tip: in CRXDE find the content node → Properties tab → check sling:resourceType.*

Session 4

Custom Components

Component anatomy, HTL, dialogs, editConfig, design dialog, inheritance, style system

Component Anatomy — 4 Required Files

- `.content.xml` — registers the component: jcr:title, componentGroup, sling:resourceSuperType
- `card.html` — HTL rendering template (filename MUST match folder name)
- `_cq_dialog/.content.xml` — the Configure dialog (fields save to JCR)
- Sling Model Java class — reads JCR properties; exposes getters for HTL

Watch out: *componentGroup must exactly match what is allowed in the template policy. One typo = component invisible in the browser. Case-sensitive.*

.content.xml — Component Registration

```
jcr:root jcr:primaryType=cq:Component
jcr:title="Card"
componentGroup="My Site - Content"
sling:resourceSuperType="core/wcm/components/teaser/v2/teaser"
/>
```

Dialog — name='./property' Rule

Every dialog field has `name="./title"`. The `./` prefix means 'relative to this component's content node'. So `name="./title"` saves as JCR property `title` on the component node. Your `@ValueMapValue private String title;` reads that exact property.

_cq_editConfig — Author Toolbar Behaviour

- `cq:actions` — buttons in the blue bar: edit, delete, insert, copymove
- `cq:emptyText` — placeholder shown when component has no content
- `afteredit=REFRESH_SELF` — refresh only this component after save (faster than REFRESH_PAGE)
- `cq:dropTargets` — allow dragging DAM assets directly onto the component

Inheritance — sling:resourceSuperType

Set `sling:resourceSuperType` in your component's `.content.xml` to inherit from another component. Sling checks your folder first; if a file is missing it walks up to the parent and uses that file. You only create the files that differ from the parent.

> ***Note:*** *Extending Core Components: set sling:resourceSuperType to core/wcm/components/teaser/v2/teaser. You get all accessibility, image handling, and policies for free. Only override what you need to change.*

Style System

- **Developer** — adds CSS classes in clientlib: `.card--featured { border: 2px solid gold; }`
- **Template author** — creates named styles in policy: 'Featured' → class `card--featured`
- **Editor** — picks 'Featured' from paintbrush icon; AEM adds the class to the wrapper div

Session 5

Touch UI Dialogs

Granite UI, dialog structure, all field types, data source, multifield

Granite UI — The Dialog Technology Stack

CoralUI (Adobe's design system) → Granite UI (component library) → your dialog XML (uses Granite sling:resourceTypes) → rendered dialog in the editor.

Every field in a dialog is a Granite UI component with its own `sling:resourceType`. All paths start with `granite/ui/components/coral/foundation/form/`.

Dialog Node Tree Structure

```
_cq_dialog/ ← sling:resourceType=cq/gui/components/authoring/dialog
■■■ content ← granite/…/foundation/tabs (or container)
■■■ items
■■■ tab1 ← jcr:title='Content', sling:resourceType=…/container
■ ■■■ items ← your fields (textfield, pathfield, select…)
■■■ tab2 ← jcr:title='Style'
```

All Field Types

- `form/textfield` — single-line text input
- `form/textarea` — multi-line text input
- `form/pathfield` — path browser; `rootPath=/content` for pages, `/content/dam` for assets
- `form/select` — dropdown with fixed options (defined as child items nodes)
- `form/checkbox` — boolean toggle; `value='true'`, `uncheckedValue='false'`
- `form/numberfield` — numeric input with min/max/step
- `form/datepicker` — calendar picker; type=date|time|datetime
- `form/multifield` — repeating group of fields (see below)
- `form/tags` — tag taxonomy picker

Data Source — Dynamic Dropdown Options

When dropdown options come from live data (JCR, API, user accounts), replace the static node with a node pointing to a Sling Servlet. The servlet implements `SlingSafeMethodsServlet`, builds a list of `ValueMapResource` objects (each with 'text' and 'value' properties), and sets a `SimpleDataSource` on the request.

Multifield — Repeating Groups

The multifield component lets editors add/remove/reorder items. Each item has the same set of sub-fields. With `composite={Boolean}true`, each item is stored as a child JCR node under the container:

```
jcr:content/root/faqblock/items/
item0/ question='What is AEM?' answer='Adobe Experience Manager'
item1/ question='What is HTL?' answer='HTML Template Language'
```

Reading in Java: inject `@SlingObject private Resource currentResource;` then call `currentResource.getChild("items")` and iterate `itemsNode.getChildren()`, adapting each child to a Sling Model.

Session 6

HTL / Sightly — Complete Reference

All 13 block statements, expression language, contexts, Use API

Why HTL Replaced JSP

- JSP mixed Java scriptlets directly into HTML — hard to read, impossible to unit test
- JSP had no auto-escaping — developers forgot `encodeForHTML()` → XSS vulnerabilities
- JSP was invalid HTML — design tools couldn't process it

HTL solves all three: no Java in templates, auto-escaping by default, valid HTML5.

Expression Syntax

```
${model.title} ← simple property (auto-escaped)
${'Hello, ' + model.name} ← string concatenation
${model.count > 0 ? 'yes' : 'no'} ← ternary
${model.title || 'Default'} ← null coalescing (OR returns first truthy value)
${model.link @ context='uri'} ← context option: URL-encodes AND blocks javascript: URIs
${'Buy Now' @ i18n} ← translate through AEM i18n
```

All 13 Block Statements

- `data-sly-use.model` — load a Sling Model or JS Use object
- `data-sly-text` — set element text content (replaces children)
- `data-sly-attribute.href` — set an HTML attribute dynamically
- `data-sly-test` — conditional render; element removed if falsy
- `data-sly-list.item` — loop over a collection; `itemList.count/first/last/odd/even`
- `data-sly-repeat.item` — like list but repeats the element itself
- `data-sly-set.myVar` — define a local variable
- `data-sly-include` — inline another HTL file as a partial
- `data-sly-resource` — render a JCR sub-resource using its own resourceType
- `data-sly-template` — define a named reusable block
- `data-sly-call` — invoke a defined template with parameters
- `data-sly-element` — change the HTML tag name dynamically
- `data-sly-unwrap` — remove the wrapper element, keep children

Context (XSS Safety) — Critical Rules

- `text` — auto-detected between tags; HTML-encodes < > &
- `uri` — REQUIRED for href/src; blocks javascript: protocol

- `html` — for RTE output only; never for user input
- `styleToken` — for CSS class names; allows only valid identifier characters

> ***Watch out:*** *Always use context='uri' on href and src attributes. Missing it allows javascript: XSS attacks through editor-entered URLs.*

The Tag

The element is always removed from the HTML output. It exists only to carry `data-sly-*` attributes when you do not want a real HTML element as the wrapper. Example: includes footer content without any wrapper element.

Java Use API — Sling Models

HTL calls `data-sly-use.model='com.mysite.core.models.CardModel'`. AEM calls `request.adaptTo(CardModel.class)`. Sling finds `CardModelImpl` (matching resourceType + adapters). Injects all annotated fields. Calls `@PostConstruct`. Returns the instance bound to `model`.

HTL → Java naming: `${model.title}` → `getTitle()` | `${model.hasLink}` → `isHasLink()`

Session 7

Sling Models — Complete Reference

Every annotation, lifecycle, interface+impl pattern, testing, errors

What is a Sling Model

A Sling Model is a Java class annotated with `@Model` that automatically reads data from the JCR when a component renders. It is the Model in AEM's MVC: Java reads data, HTL renders HTML, Sling controls the flow.

@Model Annotation — All Four Parameters

- `adaptables` — `{Resource.class, SlingHttpServletRequest.class}` — use both; Request gives access to @ScriptVariable fields
- `adapters` — `{CardModel.class}` — links impl to interface; required for adaptTo() to work
- `resourceType` — `"mysite/components/card"` — auto-selects this model; must exactly match sling:resourceType (no /apps prefix, case-sensitive)
- `defaultInjectionStrategy` — `OPTIONAL` (recommended) injects null if property missing; REQUIRED throws exception

All Injectors

- `@ValueMapValue` — reads JCR property matching field name; types: String, boolean, long, double, Calendar, String[]
- `@ValueMapValue(name='jcr:title')` — override property name for special chars like colons
- `@Default(values='Read More')` — fallback when property not set
- `@SlingObject` — injects Resource, ResourceResolver, Request, Response, BundleContext
- `@ScriptVariable` — injects Page, PageManager, Style (currentStyle), XSSAPI — Request adaptable only
- `@ChildResource` — injects a named child JCR node as Resource or adapted to another model
- `@OSGiService` — injects a registered OSGi service; add @Optional if service may not be deployed
- `@RequestAttribute` — injects request.getAttribute(); @Named overrides property name

Watch out: *Never use @Inject in new code. Use the specific injectors above. @Inject tries all injectors blindly — slow, ambiguous, and hard to read.*

@PostConstruct — All Logic Lives Here

Called after ALL injected fields are populated. This is the only correct place for business logic. The constructor runs before injection — never put logic there.

```
@PostConstruct
```

```
protected void init() {
hasLink = StringUtils.isNotBlank(ctaLink);
themeClass = StringUtils.isNotBlank(theme) ? 'card--' + theme : '';
headingLevel = currentStyle != null ? currentStyle.get('headingLevel','h3') : 'h3';
}
```

Interface + Implementation Pattern

Interface `(CardModel.java)` — only getter signatures, no AEM imports, no annotations. In the `models/` package.

Implementation `(CardModelImpl.java)` — all @Model, all @ValueMapValue fields, @PostConstruct. In the `models/impl/` package.

Benefits: unit-testable (swap impl for mock), swappable via OSGi ranking, clean HTL (use interface class name), clear contract for other developers.

Lifecycle Order

- 1. HTL evaluates data-sly-use → calls adaptTo()
- 2. Sling Models factory finds matching impl (by resourceType + adapters)
- 3. Constructor runs (all fields still null)
- 4-8. @ValueMapValue, @SlingObject, @ScriptVariable, @ChildResource, @OSGiService injected
- 9. @PostConstruct runs — all injected values available
- 10. Model returned to HTL; getters called for each ${model.xxx}
- 11. @PreDestroy runs (if defined); model garbage collected

Unit Testing with AEM Mocks

```
@ExtendWith(AemContextExtension.class)
class CardModelImplTest {
AemContext ctx = new AemContext(ResourceResolverType.RESOURCERESOLVER_MOCK);
@BeforeEach void setUp() {
ctx.create().resource('/content/test/card',
'sling:resourceType', 'mysite/components/card',
'title', 'Test Title');
ctx.currentResource('/content/test/card');
ctx.addModelsForClasses(CardModelImpl.class);
}
@Test void testTitle() {
CardModel m = ctx.request().adaptTo(CardModel.class);
assertEquals('Test Title', m.getTitle());
}
}
```

Session 8

Templates & Page Component

Static templates, editable templates, page component creation, page properties

Static Template — .content.xml

```
<jcr:root jcr:primaryType="cq:Template"
jcr:title="Article Page"
allowedPaths="[/content/mysite(/.*)?]"
ranking="{Long}10"
sling:resourceType="mysite/components/page/article"
/>
```

The `sling:resourceType` here is the page component — the component that generates the complete HTML document (DOCTYPE, head, body).

Editable Template — JCR Structure

```
/conf/mysite/settings/wcm/templates/article-page/
jcr:content ← metadata (title, status, page component reference)
structure/ ← LAYER 1: locked components (header, footer, main container)
initial/ ← LAYER 2: pre-populated editable content
policies/ ← LAYER 3: references to policy nodes
```

Note: *Enable templates via Tools → Sites → Templates → select conf → toggle status to Enabled. Set cq:allowedTemplates on /content/mysite/jcr:content to control which templates appear.*

Page Component Files

- `base.html` — the root: , , ,
- `head.html` — title, meta, canonical, Open Graph, ClientLib CSS include
- `header.html` — global site header (nav, logo)
- `footer.html` — global site footer
- `bodyEnd.html` — ClientLib JS includes before

Always extend Core Components page:
`sling:resourceSuperType='core/wcm/components/page/v3/page'`. Only override the files that differ.

Including ClientLibs in head.html

```
<sly data-sly-use.clientlib="/libs/granite/sightly/templates/clientlib.html"/>
<!-- CSS in <head>: -->
<sly data-sly-call="${clientlib.css @ categories='mysite.site'}"/>
<!-- JS before </body> in bodyEnd.html: -->
```

```
<sly data-sly-call="${clientlib.js @ categories='mysite.site'}"/>
```

Page Properties

Opened via Page Information → Open Properties. All values saved to `jcr:content` of the page node. Read in HTL via `${pageProperties['cq:description']}`.

- **Basic tab** — title (jcr:title), tags, hide in nav, on/off time
- **Advanced tab** — language (jcr:language), redirect URL, vanity URL
- **Cloud Services** — analytics, Target, translation service connections
- **Permissions** — JCR access control list for this page

Session 9

Client Libraries

Dependencies, embed, include via HTL, minify, gzip, debug mode

What is a ClientLib

A ClientLib (Client Library Folder) is AEM's way of managing CSS and JavaScript. You organise files into categorised library nodes. AEM handles concatenation, minification, cache busting, and proxy serving automatically.

Without ClientLibs: hardcoded script tags, manual load order, no cache busting, many HTTP requests. With ClientLibs: one CSS file, one JS file, automatic versioned URLs.

.content.xml — Every Property

```
<jcr:root jcr:primaryType="cq:ClientLibraryFolder"
categories="[mysite.site]"
dependencies="[mysite.vendor]"
embed="[mysite.animations]"
allowProxy="{Boolean}true"
/>
```

- `categories` — the unique ID used in HTL includes and dependency references
- `dependencies` — other libraries that must be loaded BEFORE this one (separate files)
- `embed` — other libraries to physically MERGE into this library's output
- `allowProxy` — REQUIRED for Publish access via /etc.clientlibs/ proxy path

> ***Watch out:*** *Every ClientLib included on public pages must have allowProxy=true. Without it the Publish server returns 404 for the CSS/JS — broken layout for visitors.*

css.txt and js.txt — The File Manifest

```
#base=css ← all paths below are relative to css/ subfolder
reset.css
variables.css ← listed before files that use the variables
main.css
components.css
```

AEM concatenates files in the exact order listed. Order matters — a file that uses variables defined in another file must come AFTER that file.

dependencies vs embed

- **dependencies** — the dependency loads as a separate file; browser can cache it independently. Use for jQuery, Bootstrap — large shared libraries.
- **embed** — the embedded library's files are merged into this library's output file; one HTTP request instead of two. Use for small site-specific utilities always used together.

> ***Note:*** *Never embed large external libraries (jQuery). Separate cache entries mean the browser downloads jQuery once and reuses it across pages.*

Including in HTL

```
<sly data-sly-use.clientlib="/libs/granite/sightly/templates/clientlib.html"/>
<!-- CSS only (put in head.html): -->
<sly data-sly-call="${clientlib.css @ categories='mysite.site'}"/>
<!-- JS only (put in bodyEnd.html): -->
<sly data-sly-call="${clientlib.js @ categories='mysite.site'}"/>
<!-- Multiple categories: -->
<sly data-sly-call="${clientlib.css @ categories=['mysite.vendor','mysite.site']}"/>
```

Minification, Gzip, Debug

- **Minification** — removes whitespace, comments, shortens colour values. Reduces CSS/JS by 30-60%. Controlled by `HtmlLibraryManagerImpl` OSGi config property `minify=true`
- **Gzip** — compresses already-minified files before sending over network. 120 KB CSS → 22 KB gzipped. Set `gzip=true` in OSGi config.
- **Debug mode** — serves individual unminified files. URL parameter: `?debugClientLibs=true` (per-request, Author only). OSGi config: `debug=true` for always-on.

ClientLib debug URLs:

- `/libs/granite/ui/content/dumplibs.html` — list all ClientLibs and their categories
- `/libs/granite/ui/content/dumplibs.rebuild.html` — force-rebuild all ClientLib caches
- `/libs/granite/ui/content/dumplibs.test.html` — verify all dependency references resolve

OSGi Config — Runmode Pattern

- `config.author/…HtmlLibraryManagerImpl.cfg.json` → `{"minify":false,"debug":true,"gzip":false}`
- `config.publish/…HtmlLibraryManagerImpl.cfg.json` → `{"minify":true,"debug":false,"gzip":true}`